

JavaScript Lab — Experiments & Case Studies

Pranav Lakhe | PRN: 24070521248 | Symbiosis Institute of Technology, Nagpur

Experiment 1

Experiment Title

CSS Styling Methods: Internal, Inline, and External CSS

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Types</title>

  <!-- External CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- Internal CSS -->
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
    }
    .internal-h1 { color: blue; font-size: 35px; }
    .internal-p { color: red; font-size: 18px; }
    .internal-btn {
      background-color: #3498db;
      color: white;
      padding: 10px 20px;
      border: none;
      border-radius: 5px;
    }
  </style>
</head>
<body>

  <div style="position:absolute; top:10px; right:20px;">
    <strong>Name:</strong>Pranav Lakhe <br>
    <strong>PRN:</strong> 24070521248
  </div>

  <h2>1. Internal CSS</h2>
  <h1 class="internal-h1">Welcome to Internal CSS</h1>
  <p class="internal-p">This webpage is styled using Internal CSS.</p>
  <button class="internal-btn">Click Here</button>

  <hr>

  <h2>2. Inline CSS</h2>
  <h1 style="color:rgb(46, 250, 5); font-size:35px;">Welcome to Inline CSS</h1>
  <p style="color:rgb(97, 16, 226); font-size:18px;">This webpage is styled using Inline CSS.</p>
  <button style="background:#3498db; color:white; padding:10px 20px; border:none; border-radius:5px;">
    Click Here
  </button>

  <hr>
```

```

<h2>3. External CSS</h2>
<h1 class="external-h1">Welcome to External CSS</h1>
<p class="external-p">This webpage is styled using External CSS.</p>
<button class="external-btn">Click Here</button>

</body>
</html>

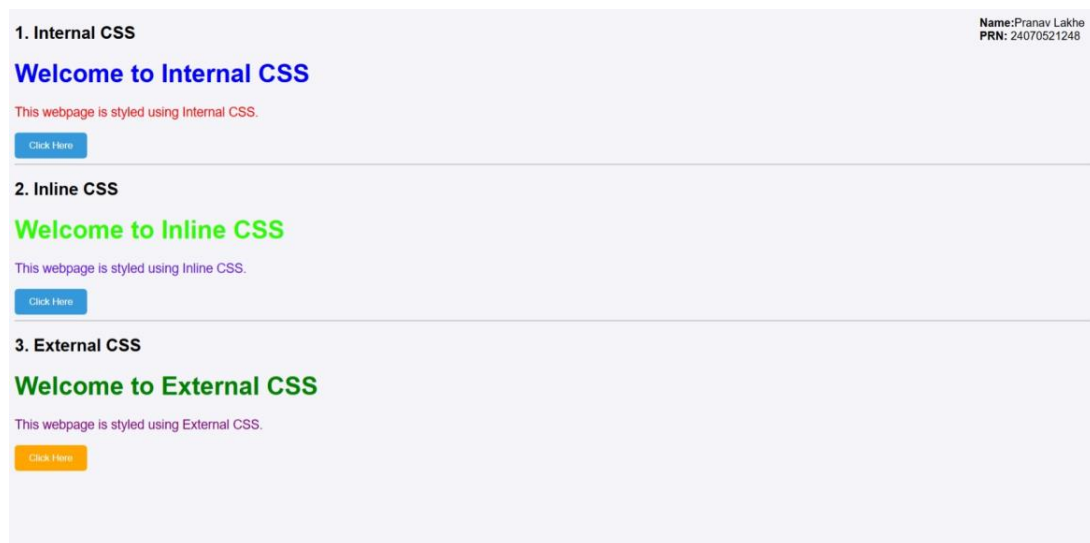
/* Style.css - External Stylesheet */
.external-h1 { color: green; font-size: 35px; }
.external-p { color: purple; font-size: 18px; }
.external-btn {
    background-color: orange;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
}

```

Output

The webpage displays three sections demonstrating CSS styling methods — Internal CSS (blue heading, red paragraph), Inline CSS (green heading, purple paragraph), and External CSS (green heading, purple paragraph, orange button) — each rendered with distinct styling to illustrate how the three approaches differ.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp1 (24070521248 Pranav Lakhe)/Exp1.html

Result/Conclusion

The three methods of applying CSS — internal, inline, and external — were successfully implemented and verified. External CSS proved most maintainable for reusable styles, inline CSS is useful for one-off overrides, and internal CSS is convenient for page-specific styling without a separate file.

Case Study 1

Case Study Title

Interactive Academic Portal: Linking College and Student Information Systems

Aim

To design and develop a modular, client-side web application using HTML5 and JavaScript that seamlessly links global institutional profiles with dynamic, individual student data validation forms.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository System
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study implements an interconnected web portal consisting of two components: College.html, the institutional landing page displaying general information, infrastructure, and courses offered with navigation to student directories; and StudentInfo.html, a dynamic information portal that captures, validates, and displays student data using client-side JavaScript logic.

The scripting layer enforces null-check filters on mandatory fields (PRN and Full Name), regex-based format matching for institutional identifiers, and onsubmit/onclick event handlers to process user entries interactively.

Case Study Program Code

College.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Academic Portal - Institutional Profile</title>
<style>
  body { font-family: Arial, sans-serif; line-height: 1.6; margin: 20px; padding: 20px; background-
color: #f4f4f9; }
  header { background: #003366; color: #fff; padding: 20px; text-align: center; border-radius: 5px; }
  section { background: #fff; margin: 20px 0; padding: 20px; border-radius: 5px; box-shadow: 0 2px 5px
rgba(0,0,0,0.1); }
  h2 { color: #003366; }
  .btn { display: inline-block; background: #003366; color: #fff; padding: 10px 20px; text-decoration:
none; border-radius: 5px; font-weight: bold; }
  .btn:hover { background: #002244; }
</style>
</head>
<body>
<header>
  <h1>Welcome to the Institutional Portal</h1>
  <p>Pranav Lakhe | Case Study 1</p>
</header>
<section id="about">
  <h2>About the Institution</h2>
  <p>This portal acts as the centralized academic hub managing campus infrastructure,
department assignments, and student demographic portfolios.</p>
</section>
<section id="departments">
  <h2>Courses & Departments</h2>
  <ul>
    <li>Department of Computer Science & Engineering</li>
    <li>Department of Information Technology</li>
    <li>Department of Data Science</li>
  </ul>
</section>
<section id="navigation">
  <h2>Student Information Directory</h2>
  <p>Access individual student verification forms, check submission integrity, and review registration
logs.</p>
</section>
</body>
</html>
```

```

    <a href="StudentInfo.html" class="btn">Open Student Information Portal &rarr;</a>
</section>
</body>
</html>

```

StudentInfo.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Information Validation Portal</title>
<style>
    body { font-family: Arial, sans-serif; line-height: 1.6; margin: 20px; padding: 20px; background-
color: #f4f4f9; }
    .container { max-width: 600px; background: #fff; margin: auto; padding: 20px; border-radius: 5px; box-
shadow: 0 2px 5px rgba(0,0,0,0.1); }
    h2 { color: #003366; text-align: center; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input[type="text"], select { width: 100%; padding: 8px; box-sizing: border-box; border: 1px solid
#ccc; border-radius: 4px; }
    button { background: #003366; color: white; padding: 10px 15px; border: none; border-radius: 4px;
cursor: pointer; width: 100%; font-size: 16px; }
    button:hover { background: #002244; }
    #outputWindow { margin-top: 20px; padding: 15px; background: #eef2f7; border-left: 5px solid #003366;
display: none; }
    .back-link { display: block; text-align: center; margin-top: 15px; color: #003366; text-decoration:
none; }
</style>
</head>
<body>
<div class="container">
    <h2>Student Registration Profile</h2>
    <form id="studentForm" onsubmit="return validateForm(event)">
        <div class="form-group">
            <label for="prn">PRN / Roll Number:</label>
            <input type="text" id="prn" placeholder="e.g., 24070521248">
        </div>
        <div class="form-group">
            <label for="fullName">Full Name:</label>
            <input type="text" id="fullName" placeholder="e.g., Pranav Lakhe">
        </div>
        <div class="form-group">
            <label for="department">Select Department:</label>
            <select id="department">
                <option value="">-- Select --</option>
                <option value="Computer Science">Computer Science</option>
                <option value="Information Technology">Information Technology</option>
                <option value="Data Science">Data Science</option>
            </select>
        </div>
        <button type="submit">Submit & Validate Profile</button>
    </form>
    <div id="outputWindow">
        <h3>Submission Profile Summary</h3>
        <p><strong>PRN:</strong> <span id="outPrn"></span></p>
        <p><strong>Name:</strong> <span id="outName"></span></p>
        <p><strong>Department:</strong> <span id="outDept"></span></p>
        <p><strong>Status:</strong> Verification Successful ✓</p>
    </div>
    <a href="College.html" class="back-link">&larr; Return to Institutional Home</a>
</div>
<script>
function validateForm(event) {
    event.preventDefault();

```

```

var prn = document.getElementById("prn").value.trim();
var fullName = document.getElementById("fullName").value.trim();
var department = document.getElementById("department").value;

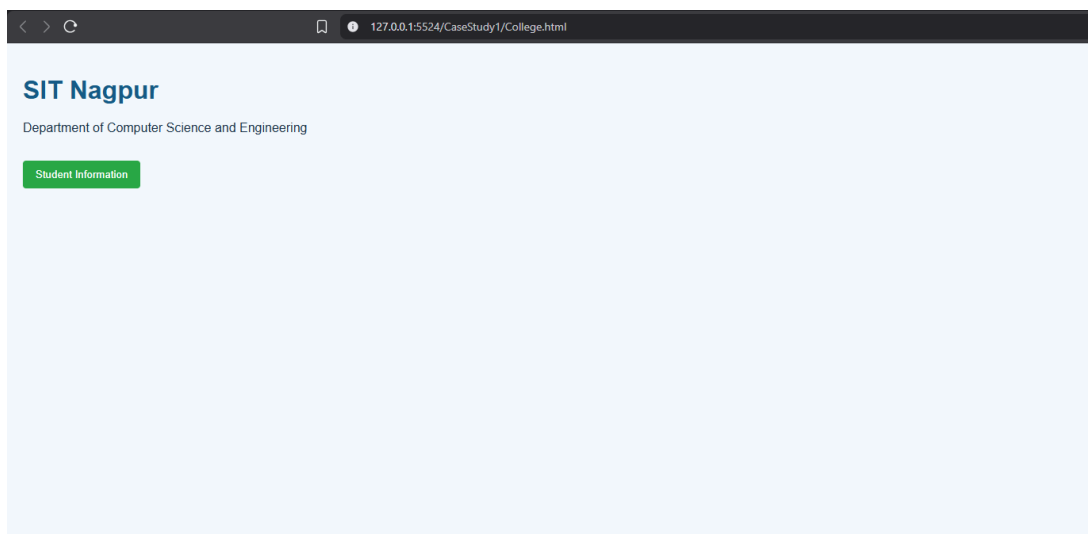
if (prn === "" || fullName === "" || department === "") {
    alert("Error: All form fields are mandatory and cannot be blank.");
    return false;
}
var prnPattern = /^[0-9]+$/;
if (!prnPattern.test(prn)) {
    alert("Error: PRN must contain numerical values only.");
    return false;
}
document.getElementById("outPrn").innerText = prn;
document.getElementById("outName").innerText = fullName;
document.getElementById("outDept").innerText = department;
document.getElementById("outputWindow").style.display = "block";
return true;
}
</script>
</body>
</html>

```

Output

The landing interface displays a clean institutional banner with department listings and a link to the student portal. On the student portal, invalid submissions trigger an alert dialog describing the specific error; valid submissions render a formatted summary profile (PRN, name, department, verification status) directly onto the page.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy1/College.html

Result/Conclusion

The interactive academic portal was successfully implemented, demonstrating page-to-page navigation, client-side form validation with null checks and regex pattern matching, and dynamic DOM updates driven entirely by JavaScript event handlers without any server-side processing.

Experiment 2

Experiment Title

Billing Calculator using ES6 Features (var, let, const, Template Literals, Destructuring)

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML Text Editor (VS Code, Sublime Text, Notepad++)
- JavaScript ES6 Support

Experiment Program Code

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Billing Calculator</title>
  <link rel="stylesheet" href="Style.css">
</head>
<body class="billing-body">
  <h2 class="title">Billing Calculator</h2>
  <div class="bill-container">
    <script>
      // Demonstrate const (immutable block-scoped)
      const GST = 0.18;

      // Demonstrate var (function-scoped/global)
      var coustomername = prompt("Enter your name:");
      var productname = prompt("Enter the name of product:");
      var quantity = Number(prompt("Enter quantity: "));
      var price = Number(prompt("Enter Price per unit:"));

      // Demonstrate let (mutable block-scoped)
      let total = quantity * price;
      let getAmount = total * GST;
      let finalAmount = getAmount + total;

      // Creating the bill object
      const bill = {
        coustomer: coustomername,
        product: productname,
        quantity,
        price,
        total,
        getAmount,
        finalbill: finalAmount
      };

      // Demonstrate Destructuring (with default value fallback)
      const {
        coustomer,
        product,
        getAmount: gstAmount = total * GST,
        finalbill
      } = bill;

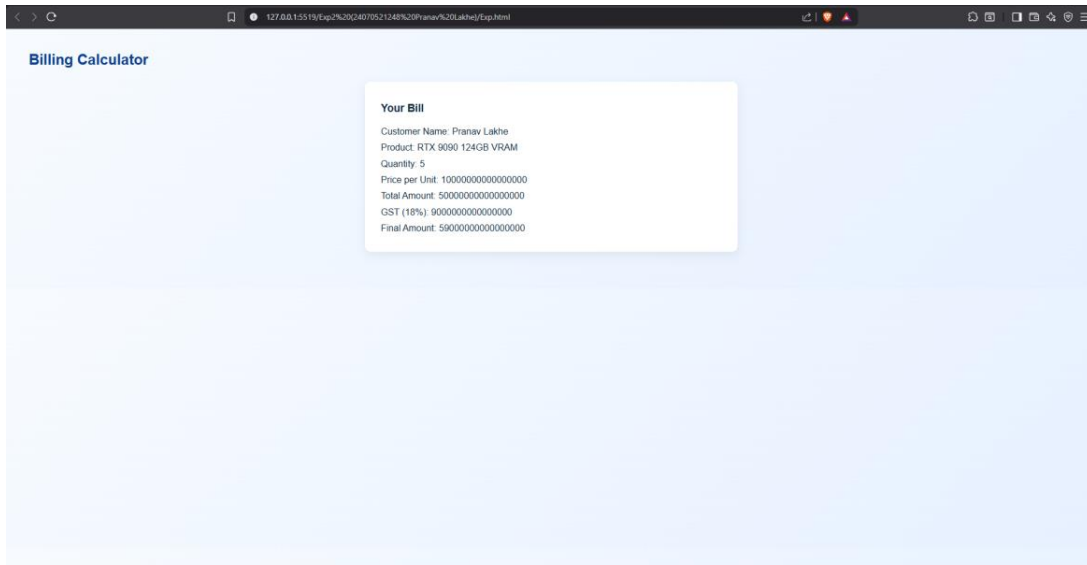
      // Demonstrate Template Literals
      document.write('<h3> Your Bill </h3>');
      document.write('<p>Customer Name: ${coustomer}</p> `');
      document.write('<p>Product: ${product}</p> `');
      document.write('<p>Quantity: ${quantity}</p> `');
      document.write('<p>Price per Unit: ${price}</p> `');
      document.write('<p>Total Amount: ${total}</p> `');
      document.write('<p>GST (18%): ${gstAmount}</p> `');
```

```
        document.write(`<p>Final Amount: ${finalbill}</p> `);
    </script>
</div>
</body>
</html>
```

Output

The calculator prompts for customer name, product name, quantity, and price per unit, then computes the total, 18% GST, and final amount, displaying a formatted bill using template literals.

Attach the program output/screenshot.



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp2 (24070521248 Pranav Lakhe)/Exp.html

Result/Conclusion

The program successfully demonstrates ES6 variable declarations (var, let, const), object destructuring with default values, and template literals for dynamic string interpolation. The bill was generated correctly using the browser prompt-based inputs.

Case Study 2

Case Study Title

Automated E-Commerce Billing Engine & Dynamic Invoice Generator

Aim

To design and develop an interactive, client-side billing application (BILL.HTML) using HTML5, CSS3, and JavaScript (ES6+) that parses real-time product quantities, applies discount thresholds, calculates taxation rates, and renders an itemised invoice directly onto the DOM.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++

- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study builds a transactional checkout application centered around BILL.HTML. It captures item pricing, quantities, and customer details, and processes them entirely on the client side — dynamically calculating subtotals, applying discount and CGST/SGST rules, and generating a printable summary instantly without server round-trips.

Key techniques include real-time numeric parsing (parseInt/parseFloat) for gross subtotals, percentage-based taxation and discount computation, and DOM display toggling to reveal the invoice receipt after submission.

Case Study Program Code

BILL.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>E-Commerce Automated Billing Engine</title>
<style>
  body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; margin: 0;
padding: 30px; background-color: #f7f9fc; }
  .billing-container { max-width: 700px; background: #ffffff; margin: auto; padding: 25px; border-
radius: 8px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
  h2, h3 { color: #1a365d; text-align: center; margin-top: 0; }
  .student-badge { text-align: center; font-size: 14px; color: #718096; margin-bottom: 25px; border-
bottom: 2px dashed #e2e8f0; padding-bottom: 10px; }
  .form-row { display: flex; gap: 15px; margin-bottom: 15px; }
  .form-group { flex: 1; display: flex; flex-direction: column; }
  label { font-weight: 600; margin-bottom: 6px; color: #2d3748; }
  input, select { padding: 10px; border: 1px solid #cbd5e0; border-radius: 5px; font-size: 15px; }
  button { background: #2b6cb0; color: #ffffff; padding: 12px; border: none; border-radius: 5px; cursor:
pointer; width: 100%; font-size: 16px; font-weight: bold; margin-top: 10px; }
  .invoice-box { margin-top: 25px; padding: 20px; background: #f8fafc; border: 1px dashed #4299e1;
border-radius: 6px; display: none; }
  .invoice-table { width: 100%; border-collapse: collapse; margin-top: 15px; }
  .invoice-table th, .invoice-table td { padding: 10px; text-align: left; border-bottom: 1px solid
#e2e8f0; }
  .total-highlight { font-size: 18px; font-weight: bold; color: #e53e3e; }
</style>
</head>
<body>
<div class="billing-container">
  <h2>E-Commerce Billing System</h2>
  <div class="student-badge">Developer: Pranav Lakhe | PRN: 24070521248</div>
  <form id="billingForm" onsubmit="generateInvoice(event)">
    <div class="form-row">
      <div class="form-group">
        <label for="customerName">Customer Name:</label>
        <input type="text" id="customerName" required>
      </div>
      <div class="form-group">
        <label for="invoiceNo">Invoice Number:</label>
        <input type="text" id="invoiceNo" placeholder="INV-2026-001" required>
      </div>
    </div>
    <div class="form-row">
      <div class="form-group" style="flex: 2;">
        <label for="itemName">Item Description:</label>
        <input type="text" id="itemName" required>
      </div>
      <div class="form-group">
        <label for="itemCategory">Category:</label>
        <select id="itemCategory">
          <option value="Electronics">Electronics (18% GST)</option>
          <option value="Clothing">Clothing (5% GST)</option>
```



```

        <option value="Essentials">Essentials (0% GST)</option>
    </select>
</div>
</div>
<div class="form-row">
    <div class="form-group">
        <label for="unitPrice">Unit Price (₹):</label>
        <input type="number" id="unitPrice" step="0.01" min="0" required>
    </div>
    <div class="form-group">
        <label for="quantity">Quantity:</label>
        <input type="number" id="quantity" min="1" required>
    </div>
    <div class="form-group">
        <label for="discountRate">Discount (%):</label>
        <input type="number" id="discountRate" min="0" max="100">
    </div>
</div>
<button type="submit">Compute Calculations & Generate Receipt</button>
</form>

<div id="invoiceBox" class="invoice-box">
    <h3>RETAIL INVOICE RECEIPT</h3>
    <table class="invoice-table">
        <thead>
            <tr><th>Item Description</th><th>Rate</th><th>Qty</th><th>Subtotal</th></tr>
        </thead>
        <tbody>
            <tr>
                <td id="invItem"></td><td id="invPrice"></td><td id="invQty"></td><td id="invSub"></td>
            </tr>
            <tr><td colspan="3">Discount Applied:</td><td id="invDisc"></td></tr>
            <tr><td colspan="3">GST:</td><td id="invTax"></td></tr>
            <tr><td colspan="3" class="total-highlight">Grand Total:</td><td id="invGrand" class="total-highlight"></td></tr>
        </tbody>
    </table>
</div>
</div>
<script>
function generateInvoice(event) {
    event.preventDefault();
    var price = parseFloat(document.getElementById("unitPrice").value);
    var qty = parseInt(document.getElementById("quantity").value);
    var discRate = parseFloat(document.getElementById("discountRate").value) || 0;
    var category = document.getElementById("itemCategory").value;

    var rawSubtotal = price * qty;
    var discountAmount = (discRate / 100) * rawSubtotal;
    var discountedSubtotal = rawSubtotal - discountAmount;

    var taxPercentage = 0;
    if (category === "Electronics") taxPercentage = 18;
    else if (category === "Clothing") taxPercentage = 5;
    else if (category === "Essentials") taxPercentage = 0;

    var taxAmount = (taxPercentage / 100) * discountedSubtotal;
    var grandTotal = discountedSubtotal + taxAmount;

    document.getElementById("invSub").innerText = rawSubtotal.toFixed(2);
    document.getElementById("invDisc").innerText = discountAmount.toFixed(2);
    document.getElementById("invTax").innerText = taxAmount.toFixed(2);
    document.getElementById("invGrand").innerText = grandTotal.toFixed(2);
    document.getElementById("invoiceBox").style.display = "block";
}
</script>

```

```
</body>
</html>
```

Output

On form submission, the application computes the raw subtotal (price $\times$ quantity), applies the entered discount percentage, calculates GST based on the selected category (Electronics 18%, Clothing 5%, Essentials 0%), and displays a formatted invoice table with subtotal, discount, tax, and grand total.

Attach the program output/screenshot.

The screenshot shows a web browser window with a URL bar displaying "127.0.0.1:5525/CaseStudy2/BILL.HTML". The page contains a form for calculating a bill. The form is divided into several sections:

- Product Information:** Includes input fields for "Product Name" (containing "rice"), "Quantity (in Kg)" (containing "40"), and "Rate per Kg (₹)" (containing "50").
- Charges & Discounts:** Includes input fields for "Discount (%)" (containing "7") and "GST (%)" (containing "18"). There is also a "Packing Charges (₹)" field with a value of "0".
- Payment & Membership:** Includes a "Payment Mode" section with radio buttons for "Cash", "Card", and "UPI" (selected). There is also a "Membership (Yes/No)" checkbox which is checked.
- Bill Summary:** A table summarizing the calculations:

Product Cost (Qty $\times$ Rate):	₹ 2000.00
Discount (7.00%):	- ₹ 140.00
Subtotal:	₹ 1860.00
GST (18.00%):	+ ₹ 334.80
Packing Charges:	+ ₹ 0.00
Total Bill Amount:	₹ 2194.80

At the bottom of the form, there are two buttons: "Calculate Bill" (in blue) and "Clear Form" (in grey).

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy2/BILL.HTML

Result/Conclusion

The billing engine was successfully implemented, demonstrating real-time arithmetic computation, conditional tax-rate assignment, and dynamic invoice rendering entirely in client-side JavaScript — confirming the feasibility of building lightweight, serverless billing tools using core web technologies.

Experiment 3

Experiment Title

Student Grading System with Dynamic Form Processing

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge, Safari)
- HTML5 with CSS3 support
- JavaScript ES6+ support
- Google Fonts (Inter, JetBrains Mono)

Experiment Program Code

Gradebook.html implements a two-column layout: a left-side form panel for entering student name, PRN, and subject-wise marks (with dynamic add/remove rows), and a right-side transcript panel. Key logic includes:

- Input validation for student name, 11-digit PRN, and marks (0–100) using regex patterns
- Grade calculation via an if/else-if control structure (Fail, D, C, B, A, A+)
- Remark generation via a switch statement mapped to each grade
- Aggregate total, percentage, and overall grade computed with a for loop over subject entries
- Dynamic transcript rendering into the DOM with color-coded grade badges

```
function getGradeInfo(marks){
  var letter;
  if(marks <= 50) letter = "Fail";
  else if(marks <= 59) letter = "D";
  else if(marks <= 69) letter = "C";
  else if(marks <= 79) letter = "B";
  else if(marks <= 89) letter = "A";
  else letter = "A+";

  var remark;
  switch(letter){
    case "Fail": remark = "Below passing threshold."; break;
    case "D":    remark = "Passed, needs improvement."; break;
    case "C":    remark = "Satisfactory."; break;
    case "B":    remark = "Good, consistent performance."; break;
    case "A":    remark = "Very good performance."; break;
    case "A+":   remark = "Outstanding performance."; break;
  }
  return { letter: letter, remark: remark };
}

// Full source: Experiments/Exp3 (24070521248 Pranav Lakhe)/Gradebook.html
```

Output

Submitting the form generates an official-style academic transcript showing student name, PRN, date of issue, subject-wise marks and grades, aggregate total, final percentage, overall grade, and Pass/Fail result with a registrar approval section.

Attach the program output/screenshot.

No screenshot file was included in the repository folder for this experiment; the output can be verified by opening Gradebook.html in a browser, filling the form, and clicking "Calculate & Generate Report."

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp3 (24070521248 Pranav Lakhe)/Gradebook.html

Result/Conclusion

The Student Grading System was successfully implemented using vanilla JavaScript, demonstrating conditional control structures (if/else-if and switch), real-time form validation, loop-based computation, and dynamic DOM manipulation to produce a professional, print-friendly transcript.

Case Study 3

Case Study Title

Automated Academic Scholarship Evaluation & Eligibility Matrix Engine

Aim

To design and implement a client-side decision-logic web application (ScholorshipEligibility.html) using HTML5, CSS3, and JavaScript conditional structures that parses academic scores, annual family income thresholds, and demographic parameters to determine student scholarship eligibility instantly.

Software/Tools Required

- Frontend Structure: HTML5
- Client-Side Scripting: JavaScript (ES6+)
- Styling Language: CSS3
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Any modern web browser (Chrome, Firefox, Edge)

Project Description

This case study implements a rule-based engine within ScholorshipEligibility.html. The system collects academic performance marks, family income, and scheme selection, then evaluates nested conditional logic against three scholarship schemes (Merit, EBC/Need-based, and Special Talent) without any server-side transaction.

Key techniques include nested conditional trapping (checking score thresholds before income limits), compound logical conditions (&&) to require both criteria simultaneously, and dynamic status-class injection to visually distinguish approved and rejected outcomes.

Case Study Program Code

ScholorshipEligibility.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Academic Scholarship Eligibility Portal</title>
<style>
  body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; margin: 0;
padding: 30px; background-color: #f0f4f8; }
  .scholarship-container { max-width: 650px; background: #ffffff; margin: auto; padding: 30px; border-
radius: 8px; box-shadow: 0 4px 20px rgba(0,0,0,0.08); }
  h2, h3 { color: #1a365d; text-align: center; margin-top: 0; }
  .developer-badge { text-align: center; font-size: 14px; color: #4a5568; margin-bottom: 25px; border-
bottom: 2px solid #e2e8f0; padding-bottom: 12px; }
  .form-group { display: flex; flex-direction: column; margin-bottom: 18px; }
  label { font-weight: 600; margin-bottom: 6px; color: #2d3748; }
  input, select { padding: 10px; border: 1px solid #cbd5e0; border-radius: 5px; font-size: 15px; }
  button { background: #4c51bf; color: #ffffff; padding: 12px; border: none; border-radius: 5px; cursor:
pointer; width: 100%; font-size: 16px; font-weight: bold; }
  .result-panel { margin-top: 25px; padding: 20px; border-radius: 6px; display: none; }
  .approved { background-color: #f0fff4; border: 1px solid #38a169; color: #276749; }
  .rejected { background-color: #fff5f5; border: 1px solid #e53e3e; color: #9b2c2c; }
</style>
</head>
<body>
<div class="scholarship-container">
  <h2>Scholarship Eligibility Portal</h2>
  <div class="developer-badge">Developer: Pranav Lakhe | PRN: 24070521248</div>
  <form id="eligibilityForm" onsubmit="evaluateEligibility(event)">
    <div class="form-group">
```

```

        <label for="studentName">Student Full Name:</label>
        <input type="text" id="studentName" required>
    </div>
    <div class="form-group">
        <label for="academicScore">Academic Score (%):</label>
        <input type="number" id="academicScore" min="0" max="100" step="0.1" required>
    </div>
    <div class="form-group">
        <label for="familyIncome">Annual Family Income (₹):</label>
        <input type="number" id="familyIncome" min="0" required>
    </div>
    <div class="form-group">
        <label for="scholarshipType">Select Scholarship Scheme:</label>
        <select id="scholarshipType">
            <option value="merit">Merit-Based Scheme (Min 85%, Max ₹8,00,000)</option>
            <option value="need">EBC Financial Assistance (Min 60%, Max ₹3,00,000)</option>
            <option value="special">Special Talent Grant (Min 75%, Max ₹6,00,000)</option>
        </select>
    </div>
    <button type="submit">Execute Logic & Check Eligibility</button>
</form>
<div id="resultPanel" class="result-panel">
    <h3 id="resultTitle">Evaluation Status</h3>
    <p><strong>Candidate Name:</strong> <span id="resName"></span></p>
    <p><strong>Scheme Checked:</strong> <span id="resScheme"></span></p>
    <p id="resDetails"></p>
</div>
</div>
<script>
function evaluateEligibility(event) {
    event.preventDefault();
    var score = parseFloat(document.getElementById("academicScore").value);
    var income = parseFloat(document.getElementById("familyIncome").value);
    var scheme = document.getElementById("scholarshipType").value;
    var isEligible = false, schemeName = "", feedbackMessage = "";

    if (scheme === "merit") {
        schemeName = "Merit-Based Scholarship Scheme";
        if (score >= 85 && income <= 800000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Profile cleared the academic and economic thresholds.";
        } else {
            feedbackMessage = "Status Denied: Requires min 85% score and income within ₹8,00,000.";
        }
    } else if (scheme === "need") {
        schemeName = "EBC Financial Assistance Scheme";
        if (score >= 60 && income <= 300000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Profile qualified under financial necessity category.";
        } else {
            feedbackMessage = "Status Denied: Requires min 60% score and income within ₹3,00,000.";
        }
    } else if (scheme === "special") {
        schemeName = "Special Talent Advancement Grant";
        if (score >= 75 && income <= 600000) {
            isEligible = true;
            feedbackMessage = "Congratulations! Candidate confirmed eligible for the allowance.";
        } else {
            feedbackMessage = "Status Denied: Requires min 75% score and income within ₹6,00,000.";
        }
    }
}

document.getElementById("resScheme").innerText = schemeName;
document.getElementById("resDetails").innerText = feedbackMessage;
var panel = document.getElementById("resultPanel");
panel.style.display = "block";

```

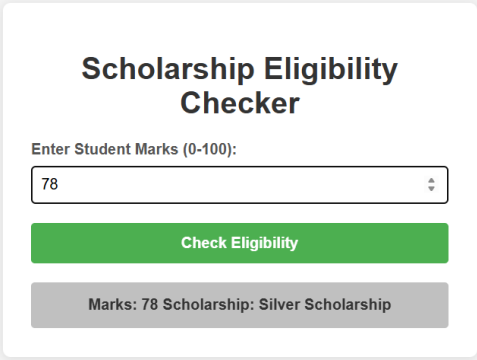
```
panel.className = isEligible ? "result-panel approved" : "result-panel rejected";
document.getElementById("resultTitle").innerText = isEligible ? "ELIGIBILITY VERIFIED ✓" :
"ELIGIBILITY REJECTED ✗";
}
</script>
</body>
</html>
```

Output

On submission, the engine checks the selected scheme's score and income thresholds and displays a result panel: green/approved with a congratulatory message if eligible, or red/rejected with the specific unmet criteria if not.

Attach the program output/screenshot.

127.0.0.1:5500/caseStudy3.html



Scholarship Eligibility Checker

Enter Student Marks (0-100):

78

Check Eligibility

Marks: 78 Scholarship: Silver Scholarship

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy3/caseStudy3.html

Result/Conclusion

The scholarship eligibility engine was successfully implemented, demonstrating multi-branch conditional logic (if/else-if), compound boolean conditions, and dynamic UI state changes based on evaluation outcomes — validating JavaScript's suitability for client-side rule-based decision systems.

Experiment 4

Experiment Title

Palindrome Checker Suite — Functions, Scope, Closures & Exception Handling

Software/Tools Required

- Web Browser (Chrome, Firefox, Edge)
- HTML5 with CSS3 support
- JavaScript ES6+ (arrow functions, const/let)
- Google Fonts (Playfair Display, Plus Jakarta Sans, JetBrains Mono)

Experiment Program Code

```
const PalindromeChecker = (function() {
  let checkCounter = 0;
  let lastExecution = null;

  function reverseString(str) {
    return str.split('').reverse().join('');
  }

  function normalizeString(str, ignorePunctuation = true, caseSensitive = false) {
    let result = str;
    if (ignorePunctuation) result = result.replace(/[^a-zA-Z0-9]/g, '');
    if (!caseSensitive) result = result.toLowerCase();
    return result;
  }

  function evaluateInput(input, options = {}) {
    if (typeof input !== 'string' || input.trim() === '') {
      throw new Error('Input must be a non-empty text string or sequence.');
```

Output

For each text or number sequence entered, the tool displays the original input, processed (normalized) form, reversed form, character length, and Palindrome/Not-Palindrome status, along with a character-alignment bubble visualization. Example: "Racecar" → Palindrome; "Hello World" → Not Palindrome.

Attach the program output/screenshot.

No screenshot file was included in the repository folder for this experiment; the output can be verified by opening `palindrome_checker.html` in a browser and using the built-in test presets (Racecar, Panama, Madam, etc.).

Student Name: Pranav Lakhe

PRN: 24070521248

File Path: Experiments/Exp4 (24070521248)/palindrome_checker.html

Result/Conclusion

The Palindrome Checker Suite successfully demonstrates closures via the IIFE module pattern (private counter and timestamp state), robust exception handling with try-catch blocks, and function scoping through helper functions. The implementation validated both text and numeric palindromes correctly, including punctuation- and case-insensitive checks.

Case Study 4

Case Study Title

Modern Multi-Theme Task Manager & Interactive To-Do Application

Aim

To design and implement a decoupled, state-driven front-end web application (`index.html`, `style.css`, `script.js`) that enables dynamic task lifecycle operations — creation, completion filtering, and deletion — alongside a live runtime theme-switching feature.

Software/Tools Required

- Frontend Structure: HTML5 Semantic Markup
- Styling & Layout: CSS3 (Custom Properties, Flexbox)
- Client Scripting: JavaScript (ES6+ Functional Modules, Event Listeners)
- Development Environment: Visual Studio Code / Notepad++
- Version Control: GitHub Repository Hub
- Execution Environment: Modern web browsers (Chrome, Edge, Firefox)

Project Description

This case study implements a three-file Task Management Board (`index.html`, `style.css`, `script.js`). Users can add tasks, toggle completion state, filter by status (All/Active/Completed), and delete entries. JavaScript manages the task array in memory, re-rendering the list on every state change, and controls a light/dark theme switch using CSS custom properties without any page reload.

Key techniques include `document.createElement()` for dynamic list-item injection, array-based state tracking with `.map()/.filter()` for status transitions and filtering, and class-based CSS variable switching for instant theme changes.

Case Study Program Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Dynamic Productivity Board</title>
<link rel="stylesheet" href="style.css">
</head>
<body class="light-theme">
```



```

<div class="app-container">
  <header class="app-header">
    <h1>Workspace Organizer</h1>
    <div class="developer-info">Developer: Pranav Lakhe | PRN: 24070521248</div>
    <button id="themeToggleBtn" onclick="toggleAppTheme()">Switch to Dark UI</button>
  </header>
  <main class="main-workspace">
    <section class="input-panel">
      <input type="text" id="taskInput" placeholder="Add a new goal or milestone..." autocomplete="off">
      <button onclick="addNewTask()">Enlist Goal</button>
    </section>
    <section class="filter-controls">
      <button class="filter-btn active" onclick="applyTaskFilter('all', this)">All Tasks</button>
      <button class="filter-btn" onclick="applyTaskFilter('pending', this)">Active</button>
      <button class="filter-btn" onclick="applyTaskFilter('completed', this)">Completed</button>
    </section>
    <ul id="taskListContainer" class="task-list"></ul>
  </main>
</div>
<script src="script.js"></script>
</body>
</html>

```

script.js

```

let taskItemsStorage = [];
let currentDisplayFilter = 'all';

function addNewTask() {
  const inputField = document.getElementById("taskInput");
  const rawText = inputField.value.trim();
  if (rawText === "") {
    alert("Please specify a task description before submission.");
    return;
  }
  taskItemsStorage.push({ id: Date.now(), text: rawText, isCompleted: false });
  inputField.value = "";
  renderTaskElementsTree();
}

function toggleTaskState(id) {
  taskItemsStorage = taskItemsStorage.map(item =>
    item.id === id ? { ...item, isCompleted: !item.isCompleted } : item
  );
  renderTaskElementsTree();
}

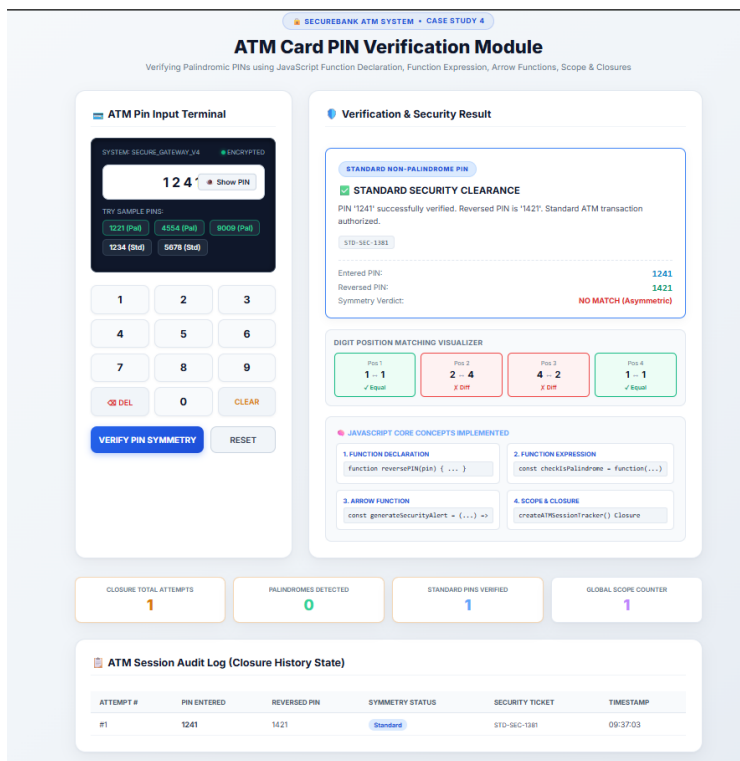
function deleteTaskRecord(id) {
  taskItemsStorage = taskItemsStorage.filter(item => item.id !== id);
  renderTaskElementsTree();
}

function applyTaskFilter(filterMode, activeBtn) {
  currentDisplayFilter = filterMode;
  document.querySelectorAll(".filter-btn").forEach(btn => btn.classList.remove("active"));
  activeBtn.classList.add("active");
  renderTaskElementsTree();
}

function renderTaskElementsTree() {
  const outputContainer = document.getElementById("taskListContainer");
  outputContainer.innerHTML = "";
  const targetedRows = taskItemsStorage.filter(item => {
    if (currentDisplayFilter === 'pending') return !item.isCompleted;
    if (currentDisplayFilter === 'completed') return item.isCompleted;
    return true;
  });

```

}



Student Name: Pranav Lakhe

PRN: 24070521248

File Path: CaseStudy4/index.html

Result/Conclusion

The task management board was successfully implemented, demonstrating array-driven state management, dynamic DOM node creation and re-rendering, event-driven filtering, and CSS custom-property-based theme switching — confirming a clean separation of concerns across structure (HTML), presentation (CSS), and behavior (JavaScript).